# BibleGoAI: Synthesis of Biblical Commentary — A Multi-Stage Pipeline with Retrieval-Augmented Generation, Large Language Models, and Semantic Similarity Scoring

Eric Hao Brandon Xia Esther Faulk Sam Faulk Mark Miller
James Reinarz David Kuo Sarah Kuo Sam Kuo Philip Tsao
Samuel Huoh Regina Graphenreed Keith Dolcy Rex Beck

## Abstract

**BibleGoAI** is a commentary-synthesis pipeline that automatically distills multi-author biblical commentary into short, accessible summary paragraphs (called *Brief Helpful Texts*, or BHTs) from a curated corpus of seventeen commentators spanning four centuries. The pipeline was run once over the full Bible to produce a fixed BHT corpus; each BHT is a short paragraph of roughly 80 words (see Section 5 for per-pipeline distributions), with automated quality scoring and provenance tracking via footnotes linking every claim back to its source commentary. The pipeline combines large language models (LLMs), retrieval-augmented generation (RAG), and semantic similarity scoring across a multi-stage flow: quote extraction, tiered synthesis, constraint-enforced generation with best-of-five selection, and embedding-based footnote linking. We evaluate the corpus across the whole Bible, comparing single-pass and split-pipeline variants on the OT. The resulting BHTs and their source commentaries are served to end users through BibleGo.org (a web application providing interactive commentary exploration, original-language word study, and full commentator browsing), reaching approximately 100–300 unique users per month. Beyond the engineering contribution, BibleGoAI is also a record of a volunteer-driven faith-community collaboration: commentator curation by ministry leaders, a 28-volunteer digitization effort of Henry Alford's 2,544-page *New Testament for English Readers* commentary, and iterative usability testing with the project's Bible study community.

## 1 Introduction

The study of the Bible has generated one of the largest and most enduring bodies of scholarly commentary in human history. Prominent theologians, from John Calvin in the sixteenth century to Archibald T. Robertson in the twentieth, have produced detailed analyses that illuminate the linguistic, theological, and spiritual dimensions of the biblical text. For many Bible studiers, these commentaries are indispensable. Yet the sheer volume of material presents a practical challenge: consulting even a handful of commentators for a single verse requires navigating multiple lengthy texts, synthesizing sometimes divergent perspectives, and extracting the most relevant insights.

BibleGoAI grew out of a simple goal: *help people study the Bible using time-tested commentaries presented in a simple and accessible way*, not only for theological understanding but also to help believers in their spiritual growth.

Modern advances in natural language processing (NLP) and large language models (LLMs) offer new possibilities for addressing this challenge. Systems based on retrieval-augmented generation (RAG) have demonstrated the ability to ground LLM outputs in authoritative source documents [Lewis et al., 2020], while sentence embeddings [Cer et al., 2018, Reimers and Gurevych, 2019] enable fine-grained semantic comparison between generated and source texts. These capabilities make it feasible to build systems that not only summarize but also *validate* the fidelity of their outputs against the original material.

In this paper, we present **BibleGoAI**, a multi-stage pipeline that transforms curated biblical commentary into concise, source-adherent summary paragraphs

called *Brief Helpful Texts* (BHTs). The name reflects the project's core value: bringing the richness of difficult-to-read commentary to the fingertips of Bible readers in an accessible way. The system is designed around three principles:

1. **Fidelity**: BHTs must be grounded in the actual words of trusted commentators, not in the LLM's own knowledge. The pipeline enforces this through hard word-proportion gates and prompt-level targets; see Section 5 for realized statistics.
2. **Conciseness**: Each BHT is a short paragraph of roughly 80 words and a few sentences (typically 2–4; see Section 5 for per-pipeline distributions), making it practical for quick reference during Bible study.
3. **Traceability**: Every claim in a BHT is linked back to its source commentary via footnotes with similarity scores, enabling readers to verify and explore further.

The pipeline was run once over the entire Bible: the full 27-book New Testament (NT; 7,957 verses) and the full 39-book Old Testament (OT; 23,145 verses). The OT was served through two complementary pipeline variants: a single-pass regular pipeline and a two-stage split pipeline (see Section 5 for details). It draws from a curated corpus of 17 unique commentators (nine NT, twelve OT, four shared) spanning four centuries and organized into three tiers. The resulting BHTs are served to end users through BibleGo.org, the project's public web application. It is used by a small but active Bible-study community of roughly 100–300 unique users per month, with monthly page-view volumes ranging from $\sim$200 to $\sim$2,500 depending on seasonal and community activity.

Our contributions are as follows:

- A multi-stage pipeline for commentary synthesis combining LLM-based quote extraction, tiered prompt engineering, and constraint-enforced generation.
- A composite quality scoring framework that integrates semantic similarity and source fidelity checks.
- An OT-specific design that uses retrieval-augmented generation to narrow multi-verse commentary, served through two pipeline variants (single-pass regular and two-stage split) covering all 39 OT books, with per-pipeline metrics reported.
- A user-facing application (BibleGo.org) serving the generated BHTs to a live Bible-study community, demonstrating the practical impact of the system.

**Project context.** BibleGoAI is a volunteer-driven collaboration among Christians, mostly based in the Cleveland, Ohio area, with backgrounds in software engineering and church ministry. The project proceeded through weekly round-table meetings in 2023, a volunteer commentary digitization effort (Henry Alford's 2,544-page *New Testament for English Readers* commentary, digitized by 28 volunteers in 2023, and Robert Govett's 627-page *Commentary on Revelation*, digitized through a similar process and completed in 2025), and an intensive April–May 2024 development sprint. The NT was publicly launched at a New Year's church conference in December 2023 and the OT at a Memorial Day church conference in 2024. Several technical decisions were shaped by this community context: the commentator corpus was curated by ministry leaders for general helpfulness in Bible study; usability testing was conducted with members of the project's Bible study community; the NASB 1995 translation was adopted after a request to use the ESV was declined by its publisher's licensing terms; NASB also provided the Strong's concordance tagging that backs the application's original-language word study; and a sustained cadence of internal demos served as the primary feedback mechanism.

## 2 Related Work

Our work draws on several intersecting areas of research.

**Abstractive and extractive summarization.** Text summarization has been a long-standing goal of NLP, with approaches ranging from extractive methods that select key sentences [Devlin et al., 2019] to abstractive methods that generate novel text conditioned on source documents [Vaswani et al., 2017]. BibleGoAI employs a hybrid approach: the quote extraction phase is extractive (selecting verbatim passages), while the BHT generation phase is abstractive (synthesizing a coherent paragraph from selected quotes). This hybrid design allows us to leverage the fluency of LLMs while maintaining explicit grounding in source material.

**Retrieval-augmented generation.** RAG systems [Lewis et al., 2020] address the hallucination problem inherent in LLMs by conditioning generation on retrieved documents. Our system applies RAG specifically to multi-verse biblical commentary where per-verse segmentation is impractical, and organizes the retrieved context into a tiered structure across multiple independent commentaries before generation: a design choice

that differentiates it from typical RAG systems that retrieve from a single corpus.

**Semantic similarity and sentence embeddings.** The Universal Sentence Encoder [Cer et al., 2018] and Sentence-BERT [Reimers and Gurevych, 2019] provide dense vector representations that enable fine-grained semantic comparison. We use these embeddings for two distinct purposes: retrieval indexing (via FAISS [Johnson et al., 2019]) and quality scoring (measuring BHT-to-commentary and BHT-to-verse similarity, the latter also reused by the footnoter to align BHT sentences with source quotes).

**AI in digital humanities and biblical studies.** The application of NLP to religious and historical texts is a growing field. Prior work has explored machine translation of biblical texts [NLLB Team, 2022], named entity recognition in historical documents, and topic modeling across theological corpora. BibleGoAI contributes to this space by focusing specifically on *commentary synthesis*: a task that requires not only understanding the source texts but also respecting the scholarly authority and interpretive nuances of individual commentators.

**Existing digital commentary platforms.** Several widely used platforms make public-domain biblical commentary available online, including BibleHub, BlueLetterBible, StudyLight, and Logos Bible Software (commercial). These systems primarily serve as *browsers* over digitized commentary: a reader selects a verse and sees the corresponding excerpts from each commentator, one commentator at a time. They do not attempt to synthesize across commentators, enforce length budgets, or link generated text back to source quotes. BibleGoAI complements rather than replaces these tools: the source corpus itself is drawn from StudyLight, and BibleGo.org links out to full commentator text for readers who want to dive deeper. The contribution here is the *synthesis layer* (producing a short, source-grounded summary that a reader can consume in seconds rather than a multi-commentator reading session), together with the provenance mechanism that keeps each synthesized claim traceable to the underlying commentary.

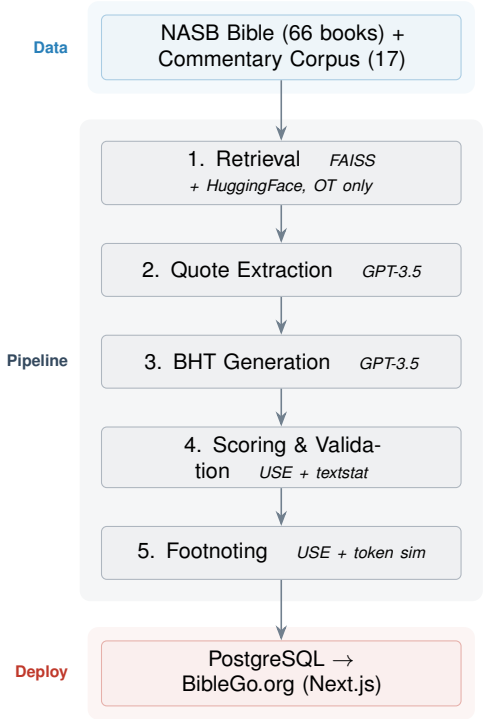


Figure 1: End-to-end system architecture. Commentary and Bible text flow through the five-phase AI pipeline, producing scored BHTs with footnotes that are stored in PostgreSQL and served via BibleGo.org. The retrieval phase is invoked only for the Old Testament route, where multi-verse commentary is narrowed to per-verse-relevant chunks before quote extraction; the New Testament route skips retrieval and feeds per-verse commentary directly to quote extraction (see Section 4).

# 3 System Architecture

BibleGoAI is composed of three major subsystems: (1) a data layer housing the biblical text and commentary corpus, (2) an AI pipeline that generates, scores, and validates BHTs, and (3) a web application that serves the results to end users. Figure 1 provides an overview of the end-to-end architecture.

## 3.1 Data Sources

The system operates on two primary data sources:

**Biblical text.** The New American Standard Bible (NASB, 1995 edition) provides the target verse text for all 66 books of the Bible. Each verse is stored as a JSON record with chapter, verse number, and text content. The NASB was selected for its reputation as a highly lit-

eral translation, which aligns well with the commentary corpus.

**Commentary corpus.** The commentary collection comprises 17 unique curated commentators (nine in the active New Testament corpus, twelve in the Old Testament corpus, with four commentators appearing in both) spanning the 16th through 20th centuries (four centuries of biblical scholarship). For the New Testament, nine commentators form the active corpus: Henry Alford, Jamieson-Fausset-Brown, Albert Barnes, Marvin Vincent, John Calvin, Philip Schaff, Archibald T. Robertson, John Gill, and John Wesley. They are organized into three tiers (Table 1). For the Old Testament, twelve commentators are used: Jamieson-Fausset-Brown, Albert Barnes, John Gill, Keil & Delitzsch, Cyrus I. Scofield, Johann P. Lange, John Calvin, William Kelly, Charles H. Mackintosh, Charles A. Coates, Arthur W. Pink, and Henri L. Rossier, similarly tiered (Table 2). Four commentators (Jamieson-Fausset-Brown, Albert Barnes, John Gill, John Calvin) appear in both the NT and OT corpora, sometimes at different tiers.

All commentaries were curated and vetted by a group of Christians from the Cleveland, Ohio area, based on their general helpfulness for Bible study.

### 3.2 Commentator Tiering

The tiering system serves a critical role in the generation process. When synthesizing a BHT, the LLM is instructed to draw *primarily* from Tier 1 commentators, *sparingly* from Tier 2, and *minimally* from Tier 3. This weighted approach ensures that the most trusted and comprehensive commentators dominate the output while allowing supplementary perspectives to enrich the synthesis. Tables 1 and 2 list the active NT and OT commentators with their tier assignments and approximate date ranges.

### 3.3 Prompt Engineering

The pipeline relies on two prompt families that shipped in production, each iteratively refined through multiple versions and recorded in the project repository: **choicest** (per-commentator quote extraction) and **BHT** (single-paragraph synthesis, with both a single-pass *regular* variant and a two-stage *split* variant comprising separate *split-a* and *split-b* sub-prompts). Two further prompt families were developed during OT exploration but did not reach the production pipeline: *tag commentary*, which attempted to segment multi-verse commentary into per-verse sections so that the unmodified NT pipeline could be reused (the segmentation proved insufficiently accurate even after fine-tuning; see Section 4); and *choicest range*, which attempted to extract single-verse quotes directly from multi-verse commentary. Verse-range handling was instead addressed through retrieval-augmented generation (Section 4.2) combined with the split pipeline for narrative OT books. The full prompt specifications, including the word-count and quote-proportion constraints used in production, appear in Sections 4.3 and 4.5.

Table 1: Active New Testament commentators and tier assignments. Tier assignments are defined per testament; several commentators (Jamieson-Fausset-Brown, Albert Barnes, John Gill, John Calvin) appear in both the NT and OT corpora and may receive different tiers.

| Commentator | Era | Tier |
|---|---|---|
| Henry Alford | 1810–1871 | 1 |
| Jamieson, Fausset & Brown | 1871 | 1 |
| Marvin R. Vincent | 1834–1922 | 1 |
| Archibald T. Robertson | 1863–1934 | 1 |
| Albert Barnes | 1798–1870 | 2 |
| Philip Schaff | 1819–1893 | 2 |
| John Wesley | 1703–1791 | 3 |
| John Gill | 1697–1771 | 3 |
| John Calvin | 1509–1564 | 3 |

Table 2: Active Old Testament commentators and tier assignments.

| Commentator | Era | Tier |
|---|---|---|
| Keil & Delitzsch | 1807–1890 | 1 |
| Arthur W. Pink | 1886–1952 | 1 |
| Henri L. Rossier | 1835–1928 | 1 |
| Charles A. Coates | 1862–1945 | 1 |
| Charles H. Mackintosh | 1820–1896 | 1 |
| Jamieson, Fausset & Brown | 1871 | 2 |
| Johann P. Lange | 1802–1884 | 2 |
| William Kelly | 1821–1906 | 2 |
| Cyrus I. Scofield | 1843–1921 | 2 |
| Albert Barnes | 1798–1870 | 3 |
| John Gill | 1697–1771 | 3 |
| John Calvin | 1509–1564 | 3 |

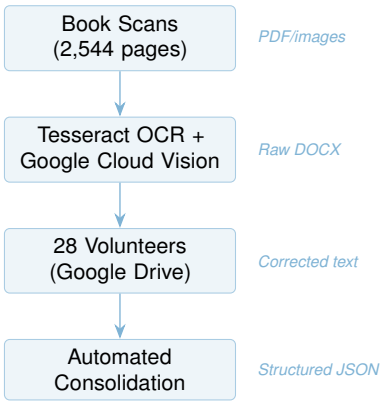


Figure 2: Commentary digitization pipeline. Numerical annotations on the diagram (page count, volunteer count, file count) refer specifically to the Alford project; Govett's *Commentary on Revelation* was digitized through the same three-stage pipeline, and per-project totals for both efforts are reported in the surrounding text.

### 3.4 Commentary Digitization

Not all commentaries in the corpus were available in digital form. Two major digitization efforts were undertaken by volunteers to expand the commentary corpus:

**Alford's New Testament for English Readers.** Henry Alford's four-volume commentary (1868–1872) was digitized from physical book scans totaling 2,544 pages across all 27 New Testament books. The pipeline (Figure 2) consisted of three stages: (1) *extraction* via Tesseract OCR and Google Cloud Vision, converting scanned pages to text; (2) *human correction*, in which 28 volunteers spent hundreds of hours reviewing and correcting OCR errors through a coordinated Google Drive spreadsheet; and (3) *automated consolidation*, which detected chapter boundaries, parsed verse-level commentary, performed spell-checking, and produced structured output files. This effort represents a significant scholarly contribution independent of the AI pipeline.

**Govett's Commentary on Revelation.** Robert Govett's commentary on Revelation was digitized through the same volunteer-driven process, spanning 627 pages and producing 274 individual verse commentary files covering all 22 chapters; this digitization effort was completed in 2025, after the BHT generation runs reported in this paper, and is therefore not part of the corpus evaluated here. Both digitization projects followed the same three-stage pipeline, demonstrating its replicability for future commentary digitization.

### 3.5 Original Language Tagging

A separate pipeline was developed to tag every word in the Bible with its corresponding Hebrew or Greek Strong's concordance number. The raw NASB text, with original-language tags provided courtesy of the NASB, was processed through a dedicated parser that produced two outputs:

- **Bible tagged files**: 66 JSON files (one per book), where each English word is linked to its Strong's code (e.g., "beginning" → H7225).
- **Tagged occurrence index**: 14,141 individual Strong's code files containing all 321,621 verse occurrences across the Bible, enabling reverse lookup from any Hebrew or Greek word to every verse where it appears.

A dictionary of 9,333 Hebrew and 5,564 Greek entries was also compiled, providing original-language forms, transliterations, and definitions. This tagging infrastructure powers the Word Donut feature on BibleGo.org (Section 7).

## 4 Methods

The BibleGoAI pipeline transforms raw multi-author biblical commentary into concise, source-adherent summary paragraphs called *Brief Helpful Texts* (BHTs). Because New Testament and Old Testament commentary tend to be organized differently—NT commentary is generally verse-by-verse, while OT commentary typically covers multi-verse passages—the pipeline runs two routes that share the same tail stages but diverge at the beginning. Figure 3 shows both routes.

### 4.1 Input Routing: NT and OT

The pipeline branches on input structure at the first stage:

- **NT route.** Commentators write verse-by-verse, so each verse's commentary can be extracted from the corpus directly and handed to the quote-extraction stage unchanged.
- **OT route.** Commentators write on multi-verse passages, so the relevant subset of each passage must first be retrieved for the target verse. Only after retrieval narrows the input does quote extraction run.

After this branching, both routes converge on the same downstream stages. Sections 4.2–4.7 describe those

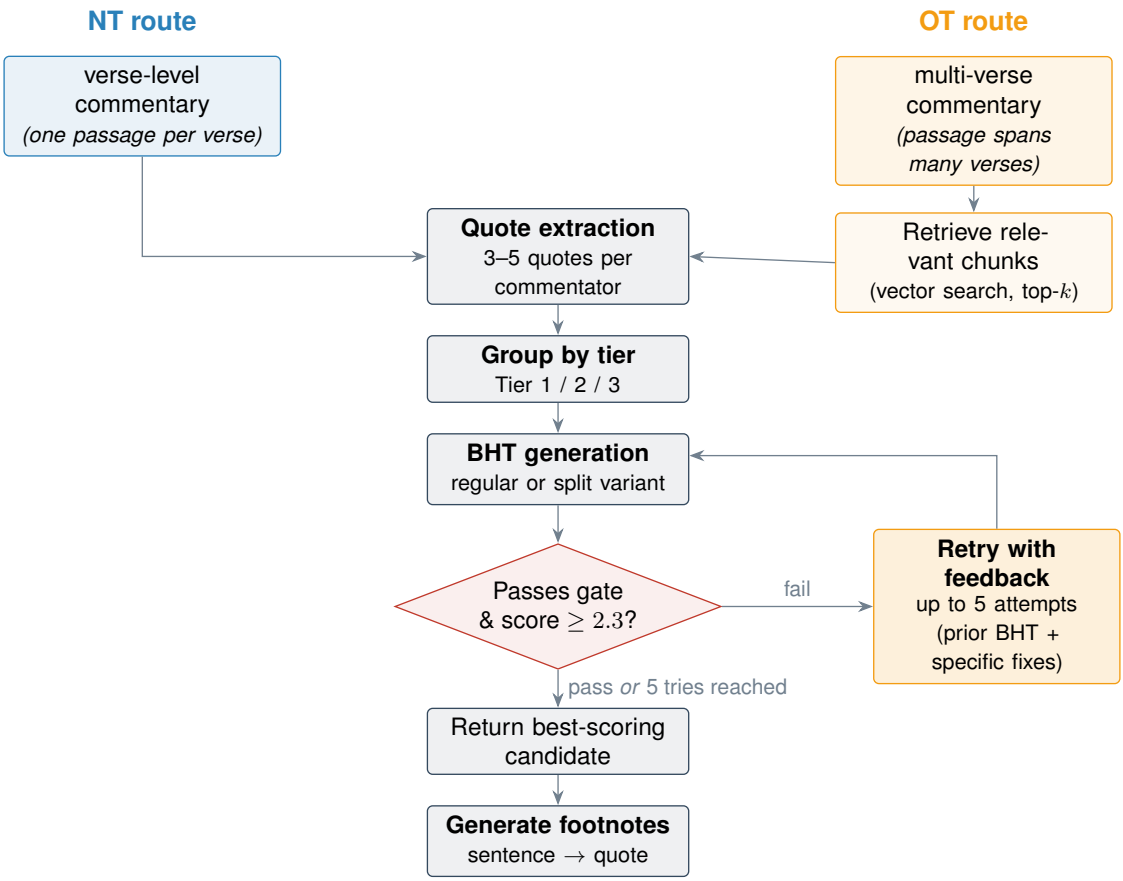


Figure 3: BHT generation flow. NT and OT inputs diverge only at the first stage: NT commentary is verse-level and is fed directly to quote extraction, while OT commentary typically covers multi-verse passages and is first narrowed to verse-relevant chunks via vector-search retrieval. The two routes then converge on a shared pipeline: quote extraction → tier grouping → BHT generation, gated by a retry loop. Retries are corrective, not blind—each failing attempt is sent back to the model together with specific complaints (word count, forbidden tokens, list formatting, etc.). The best-scoring candidate across up to five attempts is retained regardless of whether it clears the gate.

stages in order.

## 4.2 Retrieval (OT only)

Retrieval runs exclusively on the OT route. When commentary for a target verse spans multiple verses (the normal case for OT commentators), the system first retrieves the most verse-relevant chunks before extracting quotes; on the NT route, where commentary is already verse-level, this stage is skipped.

Commentary text is chunked using LangChain's `CharacterTextSplitter` [Chase, 2022] with a fixed target chunk size. Each chunk is embedded using HuggingFace's `all-mpnet-base-v2` sentence transformer [Song et al., 2020] and indexed with FAISS [Johnson et al., 2019]. At query time, the target Bible verse text is embedded and the top-$k$ most cosine-similar chunks are returned; these retrieved chunks stand in for the full commentary when quote extraction runs (Section 4.3).

Two additional chunking strategies were evaluated during OT development—LangChain's `RecursiveCharacterTextSplitter` (hierarchical splitting on paragraph/sentence/word boundaries) and `SemanticChunker` (embedding-aware, topic-respecting splits)—but were not adopted for the production corpus. An LLM-driven per-verse tagging approach was also tried as an alternative to retrieval and is described in Section 4.8.

### 4.3 Quote Extraction

Given the per-verse commentary (NT) or the retrieved chunks (OT), the system prompts GPT-3.5-turbo [OpenAI, 2023] once per commentator with a production *choicest prompt*. The NT and OT pipelines use different prompt versions with materially different extraction targets:

- **NT (v0.4):** extract 1–3 "striking, unique, and enlightening" quotes verbatim, with each quote capped at 100 words.
- **OT (v2.2):** extract 3–5 "helpful, explanatory, and illuminating" quotes verbatim that pertain specifically to the basic words and facts of the verse, with the total response capped at 500 words.

The OT version was tuned for the larger and more variable commentary chunks produced by retrieval (Section 4.2), where a single chunk can span multiple verses' worth of context, so a wider quote budget is needed to capture the relevant material. Both versions require quotes to be returned verbatim and numbered for reference.

Extracted quotes are validated against the original commentary to guard against hallucinated or paraphrased content: the set of tokens in the response is compared against the set of tokens in the source commentary, and if more than 2 tokens are out-of-vocabulary the extraction is flagged and re-requested up to 5 times. (A separate inner retry loop with up to 10 attempts handles transient API errors.) The resulting object, `ChoicestQuotesForVerse`, maps each commentator to their verified excerpts and becomes the input to the next stage.

### 4.4 Tiered Grouping

Before BHT generation, the extracted quotes are grouped by commentator tier. The NT and OT tier assignments are defined in Section 3 (Tables covering NT and OT tiering). This grouping drives two downstream behaviors: the BHT prompt's instruction to draw primarily from Tier 1, sparingly from Tier 2, and minimally from Tier 3, and the tier-weighted scoring formula (Section 4.6). The tier-steering instruction is stated explicitly in the production BHT prompt: The tier-steering instruction is stated explicitly in the production BHT prompt:

> *"You will be given the commentary quotes in three "tiers". Please focus mainly on the first-tier commentary, draw sparingly from the second-tier commentary, and only pull from the third-tier commentary when absolutely necessary."*
>
> — excerpt from `bht prompt v0.9` (OT production)

### 4.5 BHT Generation

The core synthesis step produces a BHT from the tiered quotes. Two generation modes are available:

**Regular generation.** A single call to GPT-3.5-turbo with the *BHT prompt*, which instructs the model to write a short, cohesive paragraph synthesizing the tiered quotes, with explicit per-tier weighting and a target word count. The NT (v0.8) and OT (v0.9) production prompts both target a 30–70 word, 1–3 sentence paragraph with at least 50% of words drawn from the provided commentary quotes; they differ only in minor wording around the framing of the explanation. The prompts also include the explicit per-tier instruction ("focus mainly on the first-tier commentary, draw sparingly from the second-tier commentary, and only pull from the third-tier commentary when absolutely necessary").
The prompt's stated targets are what is asked of the model; the hard-constraint gate that a candidate must clear to stop the retry loop (Section 4.6) is looser than the prompt targets in both word count and quote proportion, so the prompt biases the model toward the center of the acceptance window.

**Split-BHT variant.** During OT development, single-pass generation produced weaker outputs on long narrative passages where the model struggled to condense many quotes into a coherent paragraph. A two-stage variant was introduced to address this. Rather than splitting a single quote set in half, the two stages are given *different* commentator groupings and produce complementary halves of the final BHT:

1. **Split-A** writes a short synthesis (20–50 words, 1–3 sentences) from its assigned commentator group using the Split-A prompt.
2. **Split-B** receives the Split-A paragraph as prior context plus a *separate* commentator group, and the Split-B prompt instructs the model to *continue* the paragraph with a single additional sentence ($\leq$30 words) that extends the synthesis using quotes from the second group.

When the Split-B group contains $\leq 1$ commentator, the system falls back to regular single-pass generation. The

two halves are concatenated into the served BHT, and the split variant is scored by the same downstream metrics and competes head-to-head with the regular variant; per-variant coverage statistics are reported in Section 5.

**Corrective retry loop.** For both modes, up to 5 generation attempts are made per verse. After each attempt, the candidate is measured against the hard-constraint gate and the continuous score (Section 4.6). If any attempt clears both, the loop terminates early.

Crucially, retries are not blind re-rolls: when an attempt fails, the prior BHT is appended to the prompt as an assistant message, followed by a *complaint list* as a user message that identifies exactly which constraints were violated. For example, if a candidate exceeds the word ceiling and contains the word "commentator," the retry prompt appends the failing BHT plus specific instructions such as "Please limit your response to 80 words" and "Please do not use the word 'commentator' or 'commentary' in your response." This turns each retry into a targeted correction rather than an independent sample, biasing subsequent attempts away from the specific failure modes of previous ones.

If no attempt ever clears the gate after 5 tries, the *best*-scoring candidate (by continuous score, with hard-constraint violations as tiebreakers) is retained regardless of whether it passes—so the served corpus does contain some BHTs that fell below the stop-retry threshold.

## 4.6 Scoring and Validation

Every generated BHT is measured against a hard-constraint gate and a tier-weighted continuous score.

**Hard-constraint gate.**

- **Word count.** Regular and Split-A candidates must fall within 20–100 words; Split-B candidates must fall within 10–50 words. The prompt-level targets stated to the model are tighter (30–80 words for Regular/Split-A, 15–40 for Split-B), so the gate accepts a wider window than the prompt asks for.
- **Quote-word proportion.** 20%–90% of BHT words must originate from the provided quotes. The prompt-level target stated to the model is tighter (30–90% with a 70% target), so again the gate accepts a wider window than the prompt asks for.
- **Forbidden content.** No commentator names, verse references, or list formatting.

**Commentary-accuracy score.** The primary continuous quality metric is a tier-weighted similarity between the BHT and the source quotes, augmented by a quote-availability bonus that rewards verses with broader tier coverage. For each commentator $c$, a per-commentator score $s_c$ is the mean of $(2 \cdot \text{semantic_sim} + 1 \cdot \text{token_sim})$ across every (BHT sentence $\times$ commentator quote) pair, where semantic similarity is computed with the Universal Sentence Encoder [Cer et al., 2018] and token similarity with bag-of-words overlap. Per-commentator scores are then averaged within tier ($t_1$, $t_2$, $t_3$) and combined into a tier-weighted similarity:

$$\text{sim} = \frac{3 \cdot t_1 + 2 \cdot t_2 + 1 \cdot t_3}{W},$$

where $W$ is the sum of tier weights (3, 2, 1) restricted to tiers that had data for the verse. A verse with commentary in all three tiers is scored out of $W = 6$; a verse with only Tier 1 data is scored out of $W = 3$. This normalization makes the similarity term comparable across verses with different tier coverage. The final score combines this similarity with a quote-availability bonus that rewards verses with more commentators contributing quotes:

$$\text{score} = 2 \cdot \text{sim} + \frac{3 \cdot n_1 + 2 \cdot n_2 + 1 \cdot n_3}{19},$$

where $n_t$ is the number of commentators in tier $t$ that contributed a non-empty choicest extraction for the verse, and 19 is the maximum-coverage normalizer used in the implementation. The bonus is bounded above by 1 (when every commentator in every tier contributes), so the final score sits roughly in $[0, 3]$ where the similarity term dominates and the availability term acts as a tiebreaker that favors verses with broader source backing. The 2.3 retry-loop stop threshold and the 1.0/2.0 display-cutoff thresholds discussed below all refer to this composite score.

**Acceptance-vs-retention semantics.** A score of 2.3 is the retry-loop stopping threshold: if a candidate clears all hard constraints and scores $\geq 2.3$, the retry loop terminates on that attempt. It is not a filter on what enters the corpus—as noted above, the best-scoring candidate is retained even when no attempt reached 2.3. On the BibleGo.org reader, a separate *display-time* cutoff decides whether to surface a BHT to readers: BHTs with score $< 1.0$ are always hidden, and BHTs with score $< 2.0$ are hidden when the verse has fewer than three

total commentary sources available. Hidden BHTs are replaced with the message: *"Sorry, no help text for this verse — may the Lord give you wisdom!"*

**Single-score design.** The composite score described above is the only continuous metric used for BHT selection, retry-loop gating, and display-cutoff decisions. Within a single BHT record, it is emitted as both `commentaryAccuracyScore` and `qualityScore`. The value surfaced to the BibleGo reader as `bhtScore` is identical to these for regular BHTs; for Split-A/Split-B composite BHTs, `bhtScore` is the arithmetic mean of the Split-A and Split-B `commentaryAccuracyScores`, since no single record spans the concatenated output.

**Diagnostic metrics.** Several auxiliary metrics are logged per BHT for offline corpus analysis but are not used for selection, retry gating, or display-cutoff decisions. These include readability indices (Flesch Reading Ease, Flesch-Kincaid Grade, Gunning Fog, Automated Readability Index, Dale-Chall; Flesch 1948, Kincaid et al. 1975, Gunning 1952, Senter and Smith 1967, Dale and Chall 1948), tier-contribution percentages, and quote-token proportion. Early in scoring development we tested each of the five readability indices as a candidate component of the composite quality score; in qualitative review of the resulting BHTs, readability-weighted scoring favored more polished prose at the expense of source adherence (the design objective the pipeline was built to enforce), so the indices were retained as side statistics rather than promoted into the selection score, and were eventually dropped from active validation analysis altogether.

## 4.7 Footnoting

Once a BHT is selected, provenance links are generated between BHT content and source commentary. The footnoter reuses the same similarity computation as the commentary-accuracy score (Section 4.6): for every pair (BHT sentence $\times$ commentator's choicest quote), it computes $2 \cdot \text{semantic_sim} + 1 \cdot \text{token_sim}$, where semantic similarity is from the Universal Sentence Encoder and token similarity is bag-of-words overlap. Pairs scoring below 1.0 are discarded; the survivors are grouped by BHT sentence and sorted in descending score order, and the top entries per sentence are retained as that sentence's footnote list. Each footnote records the commentator, the quote index, the similarity score, and the byte offset where the quote appears in the commentator's original text (found via a substring matcher tolerant to minor token insertions). These annotations drive the expandable BHT sentence $\rightarrow$ source-quote navigation on BibleGo.org.

**Matthew 5:9** *("Blessed are the peacemakers: for they shall be called the children of God.")*

**Generated BHT:**

*"Blessed are the peacemakers, for they not only avoid strife but actively make peace between others. They shall be called sons of God, imitating Jesus who reconciles the separated and brings harmony to those at odds. The peacemakers are not just peaceable, but they work to diffuse peace and reconcile others. May we strive to be like them, transforming into peace-diffusers through the reconciliation and peace we have with God through Jesus Christ."*

**Pipeline metadata:**

- Commentary-accuracy score: **2.54** (above the 2.3 retry-loop threshold)
- Word count: 74 (within the 30–80 gate)
- Quote-word proportion: 48% (within the 30%–90% gate)
- Tier contributions: T1 = 32.7%, T2 = 32.4%, T3 = 34.9% (all three tiers contributed)
- Selected on generation attempt 2 of 5

**Sample footnote link.** BHT sentence 1 ("...actively make peace between others") links to Henry Alford (Tier 1) quote:

> *"They who not only are themselves not men of strife, but make peace between others when at strife."*

Figure 4: A production BHT for Matthew 5:9, with the metadata recorded by the scoring stage and an example footnote linking one BHT sentence back to its source quote. The served BHT on BibleGo.org exposes all four footnote links, one per sentence.

To make the pipeline concrete, Figure 4 shows a real BHT from the production corpus along with the metadata recorded during generation and an example footnote link.

## 4.8 Background: The OT Adaptation

The NT route was built first. Commentators in the NT corpus write verse-by-verse, so quote extraction from the full per-verse commentary worked directly and re-

quired no retrieval.

When the team turned to the OT after the public release of NT BHTs, the structural difference of OT commentary broke this assumption: OT commentators typically discuss passages spanning multiple verses, and feeding an entire multi-verse passage to the choicest prompt produced quotes that were off-topic for any specific verse. Two adaptations were explored before the current retrieval-based route stabilized:

1. **Per-verse tagging.** An experiment that asked GPT to segment multi-verse commentary into per-verse sections (via a dedicated "tag commentary" prompt), which could then feed the unmodified NT pipeline. The segmentation proved inconsistent at scale—roughly 50% accuracy on internal evaluation, where the downstream pipeline needed ∼95% to be usable (errors in tagging compound through retrieval and quote extraction, so a half-correct segmentation yields substantially worse than half-correct BHTs). Fine-tuning GPT-3.5-turbo on the tagging task via OpenAI's hosted fine-tuning API raised accuracy to roughly 60%—a measurable but nowhere-near-sufficient gain—and the tagging route was not adopted. The tagging code remains in the repository but is not invoked in the production OT run.
2. **Enhanced choicest extraction.** A modified choicest prompt that asked the model to isolate verse-specific quotes from longer passages without segmentation. This also proved unreliable—the model struggled to filter verse-relevant content from long narrative passages.

Retrieval-augmented generation (Section 4.2) resolved the problem: instead of requiring reliable segmentation, the system embeds commentary chunks and retrieves the passages most semantically relevant to each verse. This route produced usable verse-level BHTs without depending on accurate verse-by-verse structure in the source, and it is what unlocked OT coverage at corpus scale.

Because the language model used by the pipeline (GPT-3.5-turbo) is nondeterministic at generation time, re-running the pipeline on the same inputs yields different (but structurally comparable) outputs. The corpus statistics reported in Section 5 reflect a single production run; the hard-constraint gate and scoring ensure quality is reproducible in distribution even when individual outputs vary.

# 5 Evaluation

We evaluate BibleGoAI across multiple dimensions: aggregate quality metrics, pipeline comparison, generation attempt analysis, and commentator tier contribution. All metrics are computed from the full corpus of generated BHTs: 7,957 New Testament verses and the full 39-book Old Testament, which is served via two complementary pipelines: a single-pass *regular* pipeline (24 books, 10,492 verses) and a two-stage *split* pipeline (20 books, 12,653 verses) whose two stages (*split-a* and *split-b*) are concatenated into a single served BHT per verse. The two pipelines overlap at the book level on 5 books (1 Kings, Esther, Habakkuk, Hosea, Micah) but cover different verses within them, so the comparison reported below is corpus-wide over each pipeline's own book set rather than a paired per-verse ablation.

## 5.1 Aggregate Quality Metrics

Table 3 summarizes the key metrics across the NT and OT corpora. The commentary-accuracy score (identical to what is exposed in legacy JSON outputs as `qualityScore`) is a composite tier-weighted similarity between the BHT and its source quotes; quote proportion measures the fraction of BHT words drawn directly from source commentary.

The NT corpus shows higher average commentary-accuracy scores than either OT sub-corpus (1.912 vs. 1.785 / 1.822), with longer BHTs than OT regular (84 vs. 63 words) and higher quote proportions (39.6% vs. 31.8% / 32.5%). These differences are consistent with the structural contrast between the NT pipeline (verse-by-verse commentary, direct per-verse quote extraction) and the OT pipelines (multi-verse commentary, retrieval-augmented generation) described in Section 4. The two OT sub-corpora produce comparable commentary-accuracy and verse-accuracy means despite their structural differences; the principal observable difference is length (the split sub-corpus's per-verse word count is roughly 1.4× that of OT regular because the served output is a concatenation of two stages). Verse accuracy was computed only for the OT corpus; see below for discussion.

**A note on NT word count.** The observed NT mean (84.2 words) sits just above the target upper bound (80). The generation loop runs up to five attempts per verse and, if no candidate satisfies the full set of soft

Table 3: Aggregate BHT quality metrics across the three production sub-corpora: NT (single pipeline), OT regular (single-pass), and OT split (two-stage *split-a*+*split-b* concatenated per verse). The OT corpus as a whole spans 23,145 verses across all 39 books, partitioned by pipeline as shown. Commentary-accuracy and verse-accuracy scores for the split sub-corpus are per-verse means of the two stage scores; word count is the per-verse sum; quote proportion is word-count-weighted across the two stages. Values are mean ± standard deviation. Word counts are read from each BHT's stored `wordCount` field (computed by the pipeline's tokenizer at generation time); a re-computation by whitespace tokenization on the served text yields slightly lower values (e.g., 61.4 for OT regular and 83.2 for OT split-combined), reflecting tokenizer-vs-whitespace differences on contractions and punctuation.

| Metric | NT (7,957) | OT reg. (10,492) | OT split (12,653) |
|---|---|---|---|
| Comm. Acc. | $1.91 \pm 0.35$ | $1.79 \pm 0.28$ | $1.82 \pm 0.25$ |
| Word Count | $84.2 \pm 11.1$ | $62.9 \pm 11.6$ | $85.9 \pm 13.5$ |
| Quote Prop. % | $39.6 \pm 7.4$ | $31.8 \pm 6.3$ | $32.5 \pm 5.6$ |
| Verse Acc. | — | $0.42 \pm 0.15$ | $0.41 \pm 0.14$ |

constraints (30–80 words, quote-proportion, etc.), the highest-scoring attempt is retained regardless of which constraints it misses. This best-of-five fallback prioritizes commentary quality over strict length compliance, yielding a mean that slightly overshoots the 80-word upper bound; see Section 4 for the full constraint-enforcement protocol.

**Length and sentence-count compliance.** The Conciseness principle (Section 1) targets approximately 80 words and 2–4 sentences per BHT. Table 4 reports realized statistics across the four generation variants in the corpus. NT generation consistently exceeded the 1–3 sentence prompt target (mean 4.2 sentences; 50.6% of NT BHTs are exactly 4 sentences and 83.1% exceed 3 sentences), while all three OT variants stayed within or below it. The split-pipeline served output (*split-a* concatenated with *split-b*) sums to roughly 3.3 sentences per verse on average, comparable to the regular OT pipeline's 2.7. The pattern parallels the word-count finding above: prompt-level length constraints were honored on OT but silently relaxed on NT, where the best-of-five quality-first selection appears to have favored slightly longer, multi-sentence outputs over strict constraint compliance.

Table 4: Realized length and sentence-count statistics per generation variant, against a prompt-level target of approximately 80 words and 2–4 sentences (with an explicit 1–3 sentence soft target in the BHT prompts). Compliance is the percentage of BHTs in that variant landing in 1–3 sentences.

| Variant | $n$ | Words | Sent. | 1–3 sent. |
|---|---|---|---|---|
| NT v3 | 7,957 | 84.2 | 4.20 | 16.9% |
| OT regular | 10,492 | 62.9 | 2.74 | 94.3% |
| OT split-a | 12,653 | 55.1 | 2.33 | 98.5% |
| OT split-b | 12,653 | 30.8 | 1.00 | 100.0% |

**Verse accuracy (OT only).** The *verse accuracy* metric (cosine similarity between the BHT and the target-verse embedding) was computed only for the OT corpus ($0.417 \pm 0.147$). The metric is not meaningful for the NT pipeline because NT BHTs are produced from per-verse commentary directly tied to the target verse; it exists specifically as a sanity check on the OT RAG pipeline, which must select the right span across multi-verse commentary.

## 5.2 Pipeline Comparison: Regular vs. Split

Two OT generation pipelines were built, each run once over a different subset of the OT canon:

- **Regular pipeline**: A single-pass generation that produces one BHT per verse from tagged commentary, targeting 30–80 words. Run over 24 OT books (10,492 verses), predominantly wisdom, prophetic, and selected historical books where per-verse commentary coverage was adequate.
- **Split pipeline**: A two-stage generation where *split-a* produces a short factual paragraph and *split-b* adds one enriching sentence; the two outputs are concatenated per-verse into a single BHT, with a combined score computed as the mean of the two stage scores. Run over 20 OT books (12,653 verses), predominantly Pentateuch and historical narrative where verse-range commentary dominates and the split was developed to handle broader context.

**A note on the comparison.** The two pipelines were developed sequentially and run over mostly disjoint book sets. Five books (1 Kings, Esther, Habakkuk,

Hosea, Micah) appear in both pipelines' outputs, but inspection reveals that the two pipelines covered *different chapters* of these books with no overlapping verses—the regular-pipeline outputs for these books are experimental partial runs that never reached production. As a result, no paired per-verse head-to-head comparison is possible from the generated corpus; Table 5 reports corpus-wide means over each pipeline's own book set.

Table 5: Pipeline comparison across the OT corpus. Numbers are corpus-wide means over each pipeline's own book set (regular: 24 books; split: 20 books); the two sets overlap at the book level on 5 books but the regular-pipeline outputs for those books are partial experimental runs covering different verses than the split-pipeline outputs, so a paired per-verse comparison is not possible.

| **Metric** | **Regular** | **Split (combined)** |
|---|---|---|
| $n$ (verses) | 10,492 | 12,653 |
| $n$ (books) | 24 | 20 |
| Commentary Accuracy | 1.785 | 1.822 |
| Word Count | 62.9 | 85.9 |
| Quote Prop. (%) | 31.8 | 32.5 |

The two pipelines produce BHTs with comparable commentary-accuracy means (1.78 vs. 1.82) and near-identical quote-token proportions ($\sim$32%), indicating comparable source-fidelity behavior; Figure 5 visualizes the same comparison. The main observable difference is length: split-pipeline BHTs are roughly 37% longer (86 vs. 63 words) because the concatenation of *split-a* and *split-b* yields a two-paragraph output. Because the two pipelines cover different book sets, these corpus-wide means should be read as per-pipeline performance characteristics on the books each was actually run over—not as a controlled ablation.

**Stage-level view (split pipeline only).** Within the split pipeline, *split-a* produces longer, higher-scoring paragraphs on its own (mean commentary-accuracy 1.935, mean word count 55.1) while *split-b* produces shorter, lower-scoring enriching sentences (mean 1.710, mean word count 30.8). These are not alternative strategies (both stages always run together, and the combined served BHT is their concatenation), but the per-stage numbers illustrate the intended asymmetric roles of the two sub-prompts.

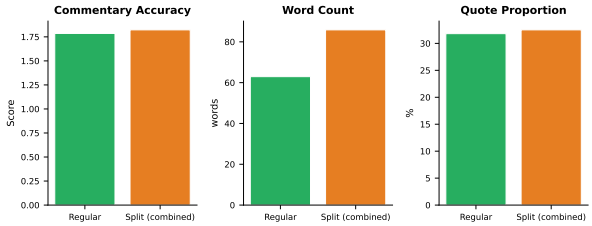


Figure 5: Corpus-wide means for the Regular and Split (combined) OT pipelines across each pipeline's own book set.

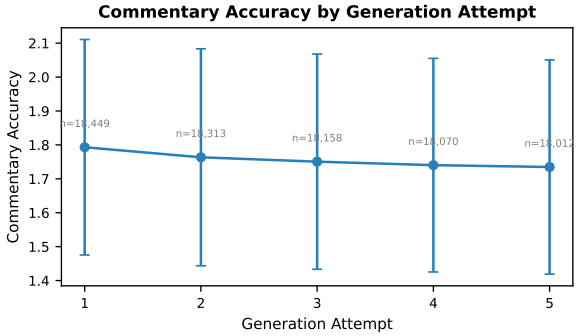


Figure 6: Mean commentary-accuracy score at each generation attempt (1–5). Scores decrease with attempt index because later attempts are only triggered when earlier ones fail the hard constraint gate, so attempts 2–5 are drawn from a harder subset of verses.

## 5.3 Generation Attempt Analysis

The system generates up to five candidate BHTs per verse and selects the highest-scoring attempt. Figure 6 shows the average commentary-accuracy score at each attempt number across all NT and OT runs combined ($n$=43,755 first attempts), which reveals a counter-intuitive pattern: mean scores *decrease* monotonically with attempt index (1.74, 1.70, 1.69, 1.68, 1.67 for attempts 1 through 5). This is not evidence that later generations are worse in absolute terms; rather, it reflects the selection dynamics of the loop: later attempts are only triggered when earlier attempts fail the hard constraint gate (word count, quote proportion, forbidden tokens), so attempts 2–5 are drawn from a harder subset of verses where the generator has already struggled. The best-of-five selection then routinely picks the first-attempt BHT. In the production run, the majority of verses succeeded on attempt 1, and the retry budget primarily functioned as safety capacity for the minority of verses where the first attempt tripped a hard constraint.

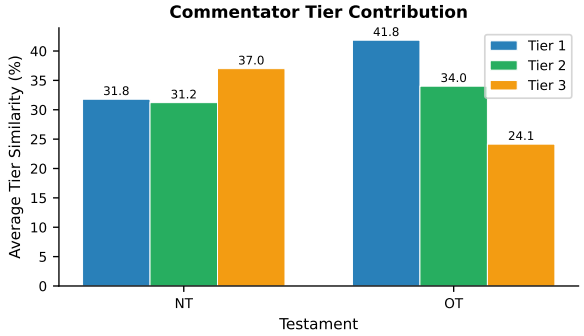


Figure 7: Average commentator-tier contribution to BHT semantic similarity, as a percentage summing to 100% per verse. NT shows a flat distribution where Tier 3 contributes at or above the level of Tier 1, indicating that the tier instruction is not strongly followed. The OT per-verse curve appears to follow the prompt ordering more cleanly, but this is driven by the split pipeline's *split-b* stage, which intentionally uses no Tier 3 commentators by construction.

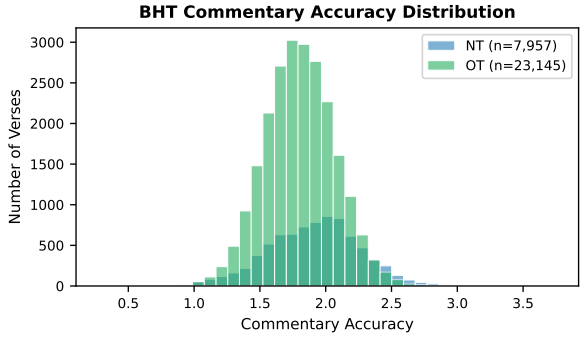


Figure 8: Distribution of commentary-accuracy scores across the NT ($n$=7,957) and OT per-verse ($n$=23,145, split-pipeline verses scored as the mean of their two stages) corpora. Both distributions are approximately Gaussian with a right-skew tail; the nominal acceptance threshold (2.3) sits near the 85th percentile of NT scores and above the 95th percentile of OT per-verse scores.

## 5.4 Commentator Tier Contribution

Figure 7 shows the distribution of tier-weighted semantic similarity across the three commentator tiers, expressed as the percentage of each tier's contribution to the final BHT's similarity profile (values sum to 100% per verse, averaged across the corpus). The NT corpus shows a flatter distribution than the prompt's "primarily Tier 1, sparingly Tier 2, minimally Tier 3" instruction would suggest: NT Tier 1 31.8%, Tier 2 31.2%, Tier 3 37.0%. The full per-verse OT corpus follows the prompt ordering more cleanly (Tier 1 41.8%, Tier 2 34.0%, Tier 3 24.1%) but is heavily influenced by the split pipeline's *split-b* stage, which intentionally uses no Tier 3 commentators (Tier 3 contribution is zero by construction in split-b). For comparison, the OT regular sub-corpus alone yields Tier 1 35.1%, Tier 2 29.7%, Tier 3 35.1%—a flatter distribution similar to NT. The NT result indicates the tier instruction is not strongly followed in the produced output. One likely cause is that several Tier 3 commentators produce longer and more thematically broad excerpts than the Tier 1 commentators, yielding higher surface-level similarity scores even when they contribute less content by intent. Reconciling the intent-based tiering with similarity-based measurement is a natural follow-up for future tuning.

## 5.5 Commentary-Accuracy Score Distribution

Figure 8 presents the distribution of commentary-accuracy scores across the NT and OT corpora. Both distributions are approximately Gaussian with a right-skew tail toward higher-scoring verses. For the NT corpus ($n$=7,957), the 5th, 25th, 50th, 75th, and 95th percentiles are 1.32, 1.68, 1.93, 2.14, and 2.46 respectively. The full per-verse OT corpus ($n$=23,145, with split-pipeline verses scored as the mean of their *split-a* and *split-b* stages) runs slightly lower: 1.37, 1.62, 1.80, 1.98, and 2.25 at the same percentiles. The retry-loop stopping threshold is a score of 2.3 (Section 4.6), which sits near the 85th percentile of NT scores and above the 95th percentile of OT per-verse scores. This threshold governs when generation stops, not which BHTs are retained: if no attempt ever clears 2.3, the best-scoring attempt is still kept (best-of-five fallback) to avoid coverage gaps, so a meaningful minority of retained BHTs fall below 2.3. A separate display-time cutoff on BibleGo.org (Section 4.6) hides very-low-scoring BHTs from readers while keeping them in the served corpus.

## 5.6 Human Evaluation

In addition to automated metrics, BHT quality was evaluated through two rounds of structured user testing with members of the project's Bible study community:

- **BHT V2 Quality Test**: Human raters scored NT BHTs on a normalized quality scale. Scores ranged

from 50.4 to 94.2 out of 100, with a mean of 67.5. The highest-rated BHT (Romans 6:7, score 94.2) demonstrated strong synthesis of commentary insights, while lower-scoring BHTs guided subsequent prompt refinements.

- **OT BHT V1 Quality Test**: A separate round evaluated the first generation of OT BHTs, providing feedback that informed the comparison of RAG versus tagging strategies.

Table 6 presents the human quality scores from the BHT V2 evaluation. Note that these are a separate metric from the automated *commentary-accuracy* score discussed above: raters scored each BHT on a normalized 0–100 quality scale.

Table 6: Human evaluation scores (BHT V2 Quality Test). Human-assigned 0–100 quality ratings; distinct from the automated commentary-accuracy score in Table 3.

| Verse | Human Score |
|---|---|
| Romans 6:7 | 94.2 |
| John 5:22 | 80.1 |
| Ephesians 1:22 | 77.3 |
| Ephesians 6:13 | 65.5 |
| Galatians 5:22 | 62.7 |
| John 10:27 | 57.0 |
| Romans 6:21 | 54.1 |
| Revelation 3:3 | 50.4 |
| **Mean** | **67.7** |

These human scores, combined with the automated quality metrics, provided a feedback loop that drove iterative prompt refinement: most notably the May 2024 prompt updates that shifted the choicest extraction focus from "striking and enlightening" to "helpful, explanatory, and illuminating" quotes targeting basic words and facts in each verse.

# 6 Discussion

**Strengths.** The multi-stage pipeline design offers several advantages over end-to-end generation. The architecture itself was shaped by an early-stage informal experiment (see *Architectural rationale* below): we initially tried prompting the model directly with raw commentary text and asking for a summary, and observed that the model consistently injected its own interpretive content alongside the commentators' positions. This failure mode—the inability to reliably distinguish *what the commentators said* from *what the model thinks they said*—drove the decision to introduce an explicit verbatim-quote extraction stage upstream of summarization, with token-set validation to enforce that the extracted quotes actually appear in the source. By extracting verbatim quotes before synthesis, the system creates an auditable intermediate representation that can be inspected and validated independently of the final BHT. The tiered commentator system ensures that the most authoritative voices dominate the output, while the constraint enforcement (word count, quote proportion, forbidden phrases) prevents many common failure modes of unconstrained LLM generation.

The iterative prompt refinement process (spanning multiple versions across the two prompt families that shipped in the production run—*choicest* and *BHT*, the latter with both a regular single-pass variant and a two-stage split variant) proved essential. Early prompt versions produced BHTs that frequently exceeded word limits, included commentator names, or adopted list formatting. Later versions largely eliminated these issues, demonstrating the value of systematic prompt engineering with quantitative feedback loops.

**Architectural rationale.** The decision to extract verbatim quotes as a discrete pipeline stage, rather than relying on the model to summarize commentary directly, was made early in the project (mid-2023) on the basis of informal experimentation across roughly 10–20 key verses, evaluated and discussed at the project's weekly round-table review. Two failure modes were consistently observed when the model was given raw commentary and asked to produce a summary: (1) the model would supplement the commentators' actual positions with its own interpretive content, often stylistically indistinguishable from the source material; and (2) when asked to quote, the model would paraphrase rather than reproduce text verbatim, even when verbatim quotation was explicitly requested. This led to the two architectural commitments that define the production pipeline: a dedicated extraction stage that forces the model to produce text claimed to come from a specific commentator, and a token-set validation check that rejects any output containing more than two tokens not present in the source. Together, these allow the multi-stage pipeline to make a stronger claim than end-to-end summarization makes by default: every fact, opinion, and turn of phrase in the final BHT is intended to be traceable to a specific

commentator's verbatim statement. We did not preserve formal records of these early experiments, and so do not present them as a quantitative ablation; we describe the reasoning here because it materially shaped the system's design.

**Limitations.** Several limitations warrant discussion. First, the entire BHT corpus was generated using GPT-3.5-turbo, which was among the most capable generally available models at the time of development; no regeneration has been performed on newer models because the pipeline was run once to produce the served corpus, and that one-time run's output quality remained acceptable while inference cost for newer models was substantially higher at the time. A full-scale regeneration and re-evaluation on more recent models remains future work. Second, the quality scoring framework, while multi-dimensional, relies heavily on embedding-based similarity, which may not capture all aspects of theological accuracy or interpretive nuance. Third, the quote-proportion constraint (30–90% of BHT words drawn from source quotes, with a 70% target), while ensuring grounding, can sometimes produce BHTs that read as stitched-together phrases rather than naturally flowing prose. Fourth, while we describe the early-stage experiments that motivated the multi-stage architecture (see *Architectural rationale* above), we do not present a quantitative same-era zero-shot ablation against GPT-3.5-turbo, nor a frontier-model zero-shot baseline using a more recent system (e.g., GPT-4-class or Claude-class models). Such ablations would help isolate the marginal contribution of the choicest-extraction stage from the model itself, and remain useful future work; we note however that any zero-shot baseline would need to be specifically engineered to satisfy the source-traceability property that the production pipeline was designed to provide, so the comparison is best understood as one of summary quality conditional on giving up (or independently re-engineering) traceability—not of summary quality alone.

**Source adherence vs. fluency.** A fundamental tension exists between source adherence and fluency of the generated text. Higher quote proportions ensure adherence but can reduce coherence; lower proportions allow smoother prose but risk introducing LLM-generated content that may not reflect the commentators' actual positions. Our operating range of 30–90% with a 70% target represents a pragmatic balance, validated through iterative testing and user feedback; the observed corpus-level means (39.6% for NT, 31.8% for OT; see Table 3) satisfy the 30% floor and stay well below the 90% ceiling, but consistently fall short of the 70% target. In practice, produced BHTs paraphrase more than the target specifies; the gap between target and observed behavior is a known limitation and a natural candidate for follow-up tuning.

**Prompt engineering insights.** The evolution of prompts yielded several practical insights: (1) explicit negative constraints ("do not mention commentators") were more effective than positive-only instructions; (2) specifying exact word count ranges reduced variance more than vague length instructions; (3) the tiering language ("primarily," "sparingly," "minimally") meaningfully influenced the tier distribution in generated outputs; and (4) requesting numbered quotes in the extraction phase significantly improved downstream validation.

# 7 Deployment: BibleGo.org

The BHTs generated by BibleGoAI are served to end users through **BibleGo.org**, a web application built with Next.js [Vercel, 2023], React, and TypeScript, backed by a PostgreSQL database and hosted on Vercel. The platform reaches approximately 100–300 unique users per month, with monthly page-view volumes ranging from $\sim$200 to $\sim$2,500 depending on seasonal and community activity. This section briefly describes the served interface so that the BHTs evaluated in Section 5 can be situated in their delivered form; full deployment details are out of scope for this paper.

**Reading interface.** The main reading interface (Figure 9) presents a two-column layout: the NASB Bible text with inline original-language tags on the left, and a commentary panel on the right. When a user selects a verse, the corresponding BHT is displayed along with footnotes that link each sentence to its source commentary; users can expand individual footnotes to view the original quotes, the commentator attribution, and a similarity score. Each commentator's full excerpt is also available as a separate card, enabling side-by-side comparison of perspectives. On mobile the commentary panel appears as a slide-up drawer (Figure 10). A separate full-commentary mode is also available, in which readers can browse the complete chapter-level text from any individual commentator.

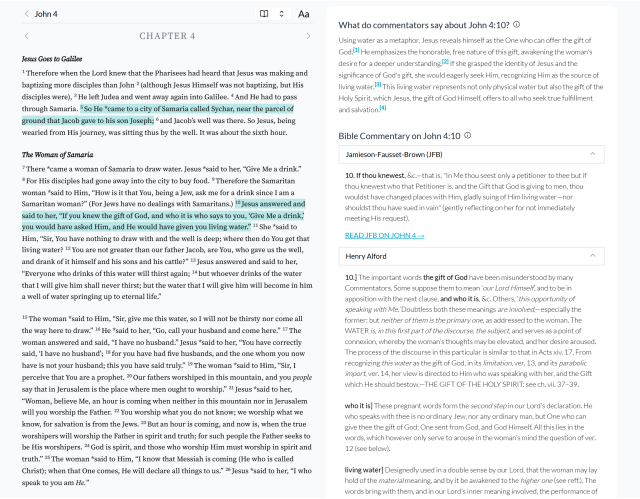


Figure 9: The BibleGo.org desktop reading interface. The two-column layout presents the NASB Bible text with inline original-language tags (left) alongside the BHT commentary panel (right), in which footnotes link each sentence back to its source commentator.

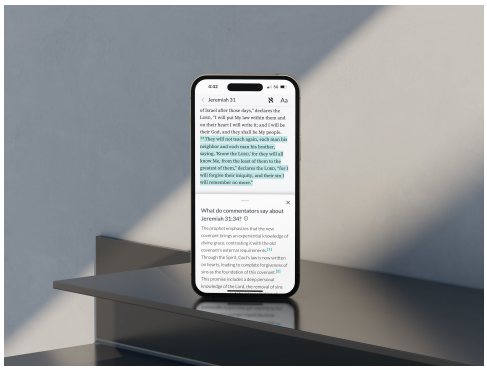

Figure 10: The BibleGo.org mobile experience with the BHT commentary drawer. Footnotes (numbered markers) link each sentence back to its source commentator, allowing users to trace every claim to the original commentary.

**Word study.** BibleGo.org also includes an interactive word-study feature, the *Word Donut*, powered by Strong's concordance data built from the original-language tagging pipeline (Section 3). A user can select any word in the Bible text to view its original Greek or Hebrew form, transliteration, and definition; an interactive doughnut chart then visualizes how the NASB translators rendered that word across all of its biblical occurrences (Figure 11). The same widget is available on mobile via the reading drawer.

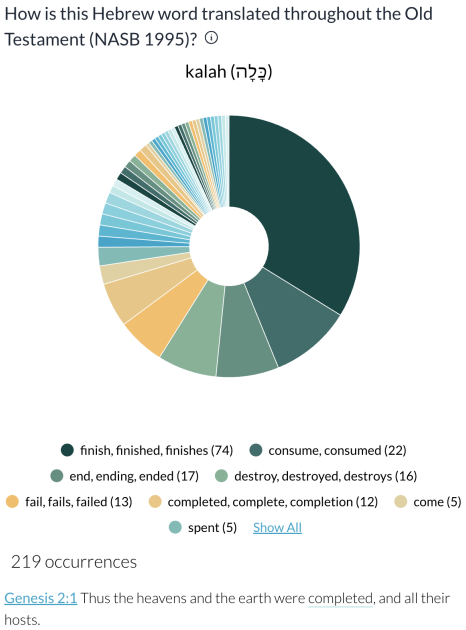


Figure 11: The Word Donut visualization for the Hebrew word *kalah*, showing how it is rendered across 221 Old Testament occurrences. Segments represent distinct English renderings (e.g., "finish" 74×, "consume" 24×), enabling readers to explore semantic range without knowledge of the original language.

**User experience and data flow.** The application provides dark mode, adjustable font sizes, mobile-responsive layout, deep linking via URL parameters (e.g., `/read?v=Matthew.1.1`), and an in-app feedback system; the UI underwent four rounds of dedicated usability testing, including tests focused specifically on the drawer navigation and the Word Donut. Pre-generated BHTs from the BibleGoAI pipeline are loaded into a PostgreSQL database and served by the Next.js application through server-side API routes; Bible text with Strong's tags is served from static JSON files, while Strong's concordance entries are queried from the database.

## 8 Conclusion and Future Work

We have presented BibleGoAI, a multi-stage pipeline for synthesizing concise, source-adherent summaries of biblical commentary using large language models, retrieval-augmented generation, and semantic similarity scoring. The pipeline was run once over a curated corpus of 17 unique commentators (nine in the NT corpus, twelve in the OT corpus, with four shared between

them) spanning four centuries, producing Brief Helpful Texts (BHTs) of roughly 80 words each (see Section 5 for per-pipeline distributions) that are grounded in source material, quality-scored across multiple dimensions, and traceable via automated footnotes. The resulting BHTs and their source commentaries are served through BibleGo.org, reaching approximately 100–300 unique users per month at the time of writing.

**Future work.** Several directions remain for future development:

- **Regeneration with stronger models**: Re-running the pipeline against the same source corpus using more recent frontier LLMs to measure how much of the residual quality gap is attributable to the GPT-3.5-era generator versus the pipeline design itself.
- **Live admin-driven regeneration**: Deploying the pipeline as a live service so that administrators can regenerate individual BHTs (or whole books) on demand as commentators are added, prompts are refined, or source-text corrections land, rather than treating BHT production as a one-shot offline process.
- **Publishing the digitized commentaries**: Releasing the digitized texts of Henry Alford's *New Testament for English Readers* and Robert Govett's *Commentary on Revelation* as standalone open resources, independent of the BHT pipeline, so the human digitization effort is reusable beyond this project.
- **Additional commentators**: Continuing to expand the curated corpus with further historical and contemporary commentators; Govett's commentary on Revelation was completed and added to the digitized corpus in 2025 following the work reported here, and similar additions are under consideration.
- **Enhanced original-language tools**: Considered for future iterations, expanding word-study features beyond Strong's concordance to include Hebrew morphology, cross-references, and parallel passage analysis.

**Lessons for similar projects.** Four patterns from this work generalize beyond biblical commentary. First, an extractive intermediate representation (the *choicest quotes* phase) proved decisive: it gives every downstream stage an auditable artifact and makes source fidelity checkable without re-reading every commentary. Second, a best-of-$N$ loop against a hard constraint gate is a low-cost way to get acceptable coverage from an imperfect generator, at the price of a small fraction of retained outputs that miss the nominal quality threshold; the attempt-level analysis in Section 5 shows this trade-off concretely. Third, a volunteer-driven curation and digitization effort can substantially expand a domain's source corpus beyond what any individual team could assemble, but only when paired with a structured consolidation pipeline that merges human correction back into machine-readable form; the same digitization pipeline served both the Alford and Govett efforts, demonstrating its replicability across projects. Fourth, automated footnote generation against the source corpus proved decisive for credibility and verifiability. Beyond the engineering value of similarity scoring, the visible per-claim footnote chain in the served BHTs gave readers a tangible verification path: the difference between "trust the LLM" and "click to read where this came from." For domains where source-claim grounding determines whether users will trust the system at all (medical, legal, theological), this is not optional polish; it is the feature that makes the system usable. Domains with similar characteristics (specialized corpora, community stewardship, and a need for claim-level traceability) may find the overall pattern reusable.

## Acknowledgments

We thank the following individuals for their contributions to commentary digitization, project direction, and user testing throughout the development of BibleGoAI and BibleGo.org: Madison Bixler, Carlf Cao, Derzan Chiang, Megan Dobos, Austin Funk, Lily Funk, Salomi Gandra, Linsu Han, Algernon Lee, Ben Sheu, Hannah Sprau, Lulu Sun, Josh Wang, and Angela Xia, along with the broader community of volunteers whose work supported the Alford and Govett digitization efforts.

## References

Daniel Cer, Yinfei Yang, Sheng-yi Kong, Nan Hua, Nicole Limtiaco, Rhomni St. John, Noah Constant, Mario Guajardo-Cespedes, Steve Yuan, Chris Tar, et al. Universal sentence encoder. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 169–174, 2018.

Harrison Chase. Langchain. https://github.com/langchain-ai/langchain, 2022.

Edgar Dale and Jeanne S. Chall. A formula for predicting readability. *Educational Research Bulletin*, 27(1): 11–20, 28, 1948.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT*, pages 4171–4186, 2019.

Rudolf Flesch. A new readability yardstick. *Journal of Applied Psychology*, 32(3):221–233, 1948.

Robert Gunning. *The Technique of Clear Writing*. McGraw-Hill, 1952.

Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.

J. Peter Kincaid, Robert P. Fishburne, Richard L. Rogers, and Brad S. Chissom. Derivation of new readability formulas for navy enlisted personnel. *Research Branch Report 8-75, Naval Technical Training Command*, 1975.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

NLLB Team. No language left behind: Scaling human-centered machine translation. In *arXiv preprint arXiv:2207.04672*, 2022.

OpenAI. GPT-3.5 turbo. https://platform.openai.com/docs/models/gpt-3-5-turbo, 2023.

Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 3982–3992, 2019.

R. J. Senter and E. A. Smith. Automated readability index. Technical Report AMRL-TR-66-220, Aerospace Medical Research Laboratories, Wright-Patterson Air Force Base, 1967.

Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. MPNet: Masked and permuted pre-training for language understanding. In *Advances in Neural Information Processing Systems*, volume 33, pages 16857–16867, 2020.

Ashwin Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.

Vercel. Next.js: The react framework for the web. https://nextjs.org/, 2023.